Trabalho Prático 01

João Gabriel Guimarães Alves Vaz

Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil
Matrícula: 2023099166
jgvaz@ufmg.br

2025

1 Introdução

O problema computacional a ser resolvido neste trabalho prático consiste em modelar, por meio de grafos, a mobilidade de pessoas em um festival de música sob chuva intensa, onde áreas seguras são representadas por abrigos circulares. Ana e Bernardo, que começaram em diferentes pontos do evento, precisam se encontrar sem sair da cobertura desses abrigos. Cada abrigo pode ser conectado a outro caso suas áreas se sobreponham ou se toquem. O desafio principal é identificar o menor número de abrigos que precisam ser atravessados para que Ana e Bernardo se reencontrem. Além disso, o sistema deve determinar o pior caso de deslocamento entre dois pontos acessíveis, isto é, o maior número de abrigos a ser cruzado em um caminho ótimo, e também identificar quais abrigos são críticos para manter a conectividade da rede de abrigos — aqueles cuja remoção isolaria partes do festival. Esses problemas envolvem algoritmos clássicos de teoria dos grafos, como busca em profundidade, cálculo de componentes conectados, caminhos mínimos e identificação de pontos de articulação.

Abaixo está um resumo do que será abordado em cada uma das sessões desse documento.

- Na sessão 1, há uma introdução ao tema e este sumário;
- Na sessão 2, há detalhes da modelagem;
- Na sessão 3, há a solução para o poblema;
- Na sessão 4, há uma análise de complexidade;
- Na sessão 5, há considerações finais e a conclusão;
- Na sessão 6, há as referências.

2 Modelagem

A modelagem do problema foi feita com o uso de grafos, onde cada vértice representa um abrigo circular e as arestas indicam interseções entre abrigos, ou seja, possibilidade de transição sem se molhar. A estrutura de dados utilizada foi uma lista de adjacência, facilitando a verificação direta de conexões entre abrigos. Para resolver as questões propostas, foram aplicados algoritmos clássicos de grafos: a busca em largura (BFS) foi usada para encontrar o caminho mínimo entre Ana e Bernardo e também para calcular o maior caminho ótimo entre dois abrigos acessíveis. Além disso, para identificar os abrigos críticos, utilizei o algoritmo de Tarjan em conjunto com a busca em profundidade (DFS), permitindo detectar de forma eficiente os pontos de articulação do grafo cuja remoção comprometeria a conectividade entre os abrigos.

3 Solução

A solução foi construída com base no algoritmo de busca em largura (BFS), que se mostrou adequado para explorar grafos de forma nivelada, garantindo caminhos mínimos em termos de número de arestas. Para resolver a Parte 1, executei uma BFS a partir do abrigo onde Ana se encontra, percorrendo os abrigos conectados até alcançar o abrigo de Bernardo, armazenando a distância mínima em número de abrigos entre eles. A Parte 2 estende essa lógica para todos os pares de abrigos conectados: para cada componente conexo, uma BFS é realizada, e o maior valor de distância entre dois abrigos acessíveis é registrado como o pior caso de deslocamento. Já na Parte 3, para identificar os abrigos críticos, utilizei o algoritmo de Tarjan aliado à busca em profundidade (DFS), o que permitiu encontrar de forma eficiente os pontos de articulação do grafo — ou seja, os abrigos cuja remoção isolaria parte da estrutura. Essa abordagem eliminou a necessidade de simulações repetidas com remoção de vértices e múltiplas BFS, garantindo uma solução mais escalável e adequada para grafos maiores.

Algorithm 1 BFS (Busca em Largura)

Require: Vértice inicial s
Ensure: Vetor de distâncias a partir de s

```
Criar fila vazia Q
Marcar todos os vértices como não visitados
visitado[s] ← verdadeiro, dist[s] ← 0
Inserir s em Q
while Q não está vazia do
    u ← remover primeiro de Q
    for cada vizinho v de u do
        if v não foi visitado then
            visitado[v] ← verdadeiro
            dist[v] ← dist[u] + 1
            Inserir v em Q
        end if
    end for
end while
return vetor dist
```

Algorithm 2 Encontrar Abrigos Críticos

Require: Grafo G com n vértices
Ensure: Lista de abrigos críticos

```
criticos ← []
for cada vértice v em G do
    Remover temporariamente o vértice v e suas arestas
    Escolher um vértice u ≠ v ainda conectado
    Executar BFS a partir de u
    if algum outro vértice não foi alcançado then
        Adicionar v a criticos
    end if
    Restaurar v e suas conexões no grafo
end for
return criticos
```

Algorithm 3 Maior Caminho Ótimo

Require: Grafo G onde vértices representam abrigos e arestas representam conexões
Ensure: Maior número de abrigos atravessados em um caminho ótimo dentro de regiões biconexas

```
biconexas ← Tarjan_Componentes_Biconexas(G)
maxDist ← 0
for cada componente C em biconexas do
    for cada vértice v em C do
        dist ← BFS(G[C], v)
        maxDist ← max(maxDist, max(dist))
    end for
end for
return maxDist
```

Algorithm 4 Tarjan Componentes Biconexas

Require: Grafo G
Ensure: Lista de componentes biconexas

```
Inicializar tempo ← 0, pilha ← [], resultado ← []
Inicializar vetores desc, low com −1 para todos os vértices
for cada vértice u não visitado do
    Chamar Tarjan_DFS(u, −1)
end for
return resultado
```

Algorithm 5 Tarjan DFS

Require: Vértice atual u, pai *parent*
Ensure: Atualiza vetores low e desc; extrai componentes biconexas

```
desc[u] ← tempo
low[u] ← tempo
tempo ← tempo + 1
filhos ← 0
for cada vizinho v de u do
    if desc[v] = −1 then
        Adicionar aresta (u, v) à pilha
        filhos ← filhos + 1
        Chamar Tarjan_DFS(v, u)
        low[u] ← min(low[u], low[v])
        if low[v] ≥ desc[u] then
            Extrair componente da pilha até (u, v)
            Adicionar componente a resultado
        end if
    else if v ≠ parent e desc[v] < desc[u] then
        Adicionar aresta (u, v) à pilha
        low[u] ← min(low[u], desc[v])
    end if
end for
```

4 Análise de Complexidade

Nesta seção, apresento a análise assintótica de tempo e memória da solução desenvolvida. A modelagem foi feita com um grafo não direcionado, onde cada vértice representa um abrigo, e as arestas são estabelecidas entre pares de abrigos cujas áreas circulares se sobrepõem ou se tocam. O grafo é representado por uma lista de adjacência. A solução resolve três subproblemas principais, cada um com suas próprias exigências algorítmicas, conforme analisado a seguir.

Parte 1: Caminho mínimo entre Ana e Bernardo

Para resolver este subproblema, implementei a *Busca em Largura* (BFS) a partir do abrigo em que Ana se encontra. A BFS percorre os níveis do grafo até encontrar o abrigo de Bernardo, registrando a quantidade mínima de abrigos atravessados.

- **Tempo:** O algoritmo BFS tem complexidade $\mathcal{O}(V + E)$, onde V é o número de vértices e E o número de arestas.
- **Memória:** A BFS utiliza um vetor de visitados $\mathcal{O}(V)$ e uma estrutura de camadas que, no pior caso, pode conter $\mathcal{O}(V)$ entradas, gerando uma complexidade espacial de $\mathcal{O}(V)$.

Parte 2: Maior caminho ótimo entre dois abrigos acessíveis

Para identificar o maior número de abrigos que uma pessoa pode precisar atravessar, executei uma BFS para cada componente conexa do grafo, e calculei a profundidade máxima atingida.

- **Tempo:** Como cada BFS tem custo $\mathcal{O}(V + E)$ e pode ser chamada até V vezes (uma por componente), a complexidade total é $\mathcal{O}(V \cdot (V + E)) = \mathcal{O}(V^2 + VE)$ no pior caso.
- **Memória:** Cada BFS consome no máximo $\mathcal{O}(V)$ de memória, como discutido na Parte 1.

Parte 3: Identificação de abrigos críticos

Nesta etapa, implementei o **Algoritmo de Tarjan**, um algoritmo em tempo linear que utiliza busca em profundidade (DFS) para encontrar pontos de articulação (ou vértices de corte) em um grafo. Um vértice é considerado crítico se sua remoção aumentar o número de componentes conexas do grafo. A escolha do Algoritmo de Tarjan é crucial aqui: uma abordagem ingênua baseada na remoção e reconstrução do grafo para cada vértice teria complexidade quadrática ou cúbica, tornando-se inviável para grandes valores de V.

Pseudocódigo (Tarjan):

Algorithm 6 Tarjan - Pontos de Articulação

Require: Grafo G, vértice u, vetor de visitados *visited*, tempo de descoberta *disc*, vetor *low*, tempo atual t, pai *parent*
Ensure: Vetor *articulation* com os vértices críticos

```
visited[u] ← verdadeiro
disc[u], low[u] ← t + 1
children ← 0
for cada vértice v adjacente a u do
    if v não visitado then
        children ← children + 1
        chamada recursiva em v
        low[u] ← min(low[u], low[v])
        if parent ≠ −1 e low[v] ≥ disc[u] then
            u é ponto de articulação
        end if
    else if v ≠ parent then
        low[u] ← min(low[u], disc[v])
    end if
end for
if parent = −1 e children > 1 then
    u é ponto de articulação
end if
```

- **Tempo:** O algoritmo de Tarjan percorre todos os vértices e arestas uma única vez, resultando em $\mathcal{O}(V + E)$ no pior caso.
- **Memória:** São utilizados vetores auxiliares para tempo de descoberta, valores *low*, vértices visitados e pontos de articulação, todos com $\mathcal{O}(V)$ de espaço, resultando em complexidade espacial total de $\mathcal{O}(V)$.

Resumo das Complexidades

Parte	Tempo	Memória
Parte 1	$\mathcal{O}(V+E)$	$\mathcal{O}(V)$
Parte 2	$\mathcal{O}(V \cdot (V+E)) = \mathcal{O}(V^2 + VE)$	$\mathcal{O}(V)$
Parte 3	$\mathcal{O}(V+E)$	$\mathcal{O}(V)$

Table 1: Resumo das complexidades assintóticas por parte da solução.

Considerações sobre escalabilidade

A escolha do Algoritmo de Tarjan foi fundamental para garantir a viabilidade computacional da Parte 3. Uma abordagem ingênua de verificar a conectividade do grafo removendo cada vértice individualmente exigiria reconstruções e múltiplas chamadas de BFS ou DFS, resultando em complexidade $\mathcal{O}(V \cdot (V+E)) = \mathcal{O}(V^2 + VE)$. Com Tarjan, resolvi a identificação de pontos críticos em tempo linear, preservando a eficiência da solução mesmo para instâncias de maior porte.

5 Considerações finais

A realização deste trabalho prático foi uma experiência enriquecedora, especialmente por envolver a aplicação direta de conceitos de grafos a uma situação realista e criativa. A parte mais tranquila foi a modelagem do problema, pois a associação entre abrigos e vértices, bem como a definição de conexões por interseção de áreas, foi intuitiva. Implementar a BFS também foi relativamente simples, por se tratar de um algoritmo clássico com comportamento previsível. No entanto, a identificação dos abrigos críticos exigiu maior atenção, pois envolveu a simulação de falhas estruturais e a análise repetida de conectividade, o que exigiu cuidados para preservar a integridade dos dados e otimizar o desempenho. No geral, o trabalho consolidou meu entendimento sobre grafos e suas aplicações práticas em contextos de conectividade e acessibilidade.

Além disso, a utilização de uma busca em profundidade (DFS) foi essencial para a implementação do algoritmo de Tarjan, empregado na identificação eficiente dos pontos de articulação do grafo. Esse algoritmo permitiu determinar, de forma otimizada e em tempo linear, os abrigos cuja remoção comprometeria a conectividade do sistema. Sua aplicação se mostrou fundamental para garantir a escalabilidade da solução, evitando abordagens de força bruta com custo quadrático que seriam inviáveis para instâncias maiores. A integração da DFS com o algoritmo de Tarjan reforçou a importância de algoritmos clássicos na análise estrutural de grafos, ampliando ainda mais minha compreensão sobre sua utilidade em cenários práticos e robustos.

6 Referências

- Almeida, J. (2025). Slides virtuais da disciplina de algoritmos 1. Disponibilizado via moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.
- Aula 9 Árvore Geradora Mínima, União-Busca, Componentes Conexos e Articulações e Pontes. Universidade Federal do Paraná, disponível aqui.